COMP30026 Models of Computation

Lecture 21 – Closure Properties of Decidable and Recognizable Languages

Cezary Kaliszyk, William Umboh, and
Peaker Guo (guest lecture)

October 14, 2026

Hello! I’m Peaker

- PhD from UniMelb; now at Institute of Science Tokyo
- I work on string algorithms and text compression

Hello! I’m Peaker

- PhD from UniMelb; now at Institute of Science Tokyo
- I work on string algorithms and text compression
- This subject comes up in my research field:

Hello! I’m Peaker

- PhD from UniMelb; now at Institute of Science Tokyo
- I work on string algorithms and text compression
- This subject comes up in my research field:
  - **Directed acyclic word graph**: a DFA that is also a data structure for text search

Hello! I’m Peaker

- PhD from UniMelb; now at Institute of Science Tokyo
- I work on string algorithms and text compression
- This subject comes up in my research field:
  - **Directed acyclic word graph** (**DAWG**): a DFA that is also a data structure for text search

Me: I got that **DAWG** in me
Actually me:



cr: IG

Hello! I'm Peaker

- PhD from UniMelb; now at Institute of Science Tokyo
- I work on string algorithms and text compression
- This subject comes up in my research field:
  - **Directed acyclic word graph** (**DAWG**): a DFA that is also a data structure for text search
  - **Grammar compression**: store a text as a CFG that generates only that text
    - LZW, used in GIF
    - Re-Pair, by N. Jesper Larsson and Alistair Moffat

Me: I got that **DAWG** in me
Actually me:



cr: IG

Hello! I'm Peaker

- PhD from UniMelb; now at Institute of Science Tokyo
- I work on string algorithms and text compression
- This subject comes up in my research field:
  - **Directed acyclic word graph** (**DAWG**): a DFA that is also a data structure for text search
  - **Grammar compression**: store a text as a CFG that generates only that text
    - LZW, used in GIF
    - Re-Pair, by N. Jesper Larsson and Alistair Moffat
  - …

Me: I got that **DAWG** in me
Actually me:



cr: IG

Quick poll #1

M recognizes L. Which statement is always true?

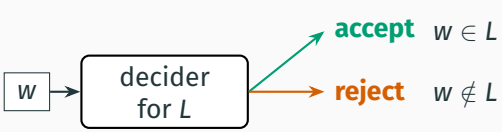
(a) If M rejects w, then $w \notin L$.

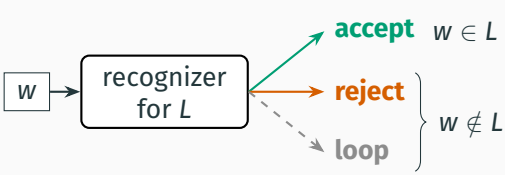
(b) If $w \notin L$, then M rejects w.



Recap: deciders and recognizers (Lecture 18)

For a language L:





Quick poll #2

$A_{\mathrm{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$

Given a TM M and a string w, does M accept w?

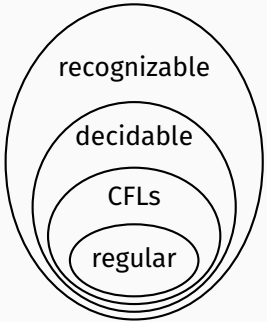
A_{TM} is ...

(a) not recognizable
(b) recognizable, but not decidable
(c) decidable



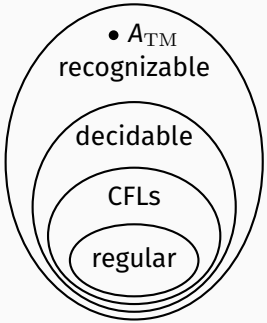
Last time (Lecture 20)

- $A_{\mathrm{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$
- A_{TM} is recognizable
- A_{TM} is not decidable



Last time (Lecture 20)

- $A_{\mathrm{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$
- A_{TM} is recognizable
- A_{TM} is not decidable



What are closure properties?

A class is *closed under* an operation if applying the operation to languages in the class always gives a language in the class.

Today's overview

Closure properties of decidable and recognizable languages:

	intersection	union	concatenation	Kleene star	complement
regular	✓	✓	✓	✓	✓
decidable	?	?	?	?	?
recognizable	?	?	?	?	?

Today's overview

Closure properties of decidable and recognizable languages:

	intersection	union	concatenation	Kleene star	complement
regular	✓	✓	✓	✓	✓
decidable	?	?	?	?	?
recognizable	?	?	?	?	**?**

Decidable languages

Decidable languages: union and intersection

For languages L_1 and L_2:

- their *union* $L_1 \cup L_2$ is the set of all strings in L_1 *or* L_2 (or both);
- their *intersection* $L_1 \cap L_2$ is the set of all strings in *both* L_1 and L_2.

Decidable languages: union and intersection

For languages L_1 and L_2:

- their *union* $L_1 \cup L_2$ is the set of all strings in L_1 *or* L_2 (or both);
- their *intersection* $L_1 \cap L_2$ is the set of all strings in *both* L_1 and L_2.

Theorem

If L_1 and L_2 are decidable, then $L_1 \cup L_2$ and $L_1 \cap L_2$ are decidable.

Decidable languages: union and intersection

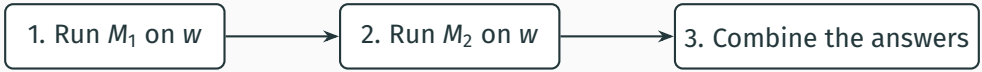
For languages L_1 and L_2:

- their *union* $L_1 \cup L_2$ is the set of all strings in L_1 *or* L_2 (or both);
- their *intersection* $L_1 \cap L_2$ is the set of all strings in *both* L_1 and L_2.

Theorem

If L_1 and L_2 are decidable, then $L_1 \cup L_2$ and $L_1 \cap L_2$ are decidable.

Let M_1 and M_2 be deciders for L_1 and L_2, respectively.



Decidable languages: union and intersection

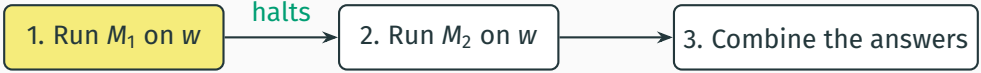
For languages L_1 and L_2:

- their *union* $L_1 \cup L_2$ is the set of all strings in L_1 *or* L_2 (or both);
- their *intersection* $L_1 \cap L_2$ is the set of all strings in *both* L_1 and L_2.

Theorem

If L_1 and L_2 are decidable, then $L_1 \cup L_2$ and $L_1 \cap L_2$ are decidable.

Let M_1 and M_2 be deciders for L_1 and L_2, respectively.



Decidable languages: union and intersection

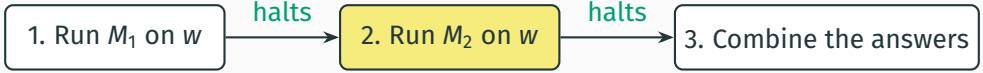
For languages L_1 and L_2:

- their *union* $L_1 \cup L_2$ is the set of all strings in L_1 *or* L_2 (or both);
- their *intersection* $L_1 \cap L_2$ is the set of all strings in *both* L_1 and L_2.

Theorem

If L_1 and L_2 are decidable, then $L_1 \cup L_2$ and $L_1 \cap L_2$ are decidable.

Let M_1 and M_2 be deciders for L_1 and L_2, respectively.



Decidable languages: union and intersection

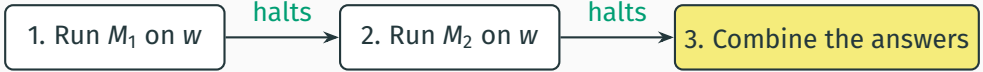
For languages L_1 and L_2:

- their *union* $L_1 \cup L_2$ is the set of all strings in L_1 *or* L_2 (or both);
- their *intersection* $L_1 \cap L_2$ is the set of all strings in *both* L_1 and L_2.

Theorem

If L_1 and L_2 are decidable, then $L_1 \cup L_2$ and $L_1 \cap L_2$ are decidable.

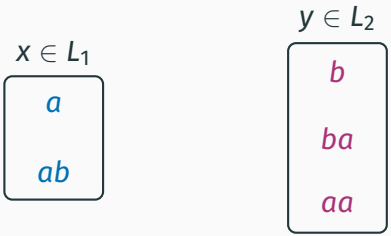
Let M_1 and M_2 be deciders for L_1 and L_2, respectively.



union: accept if *either* accepted
intersection: accept if *both* accepted

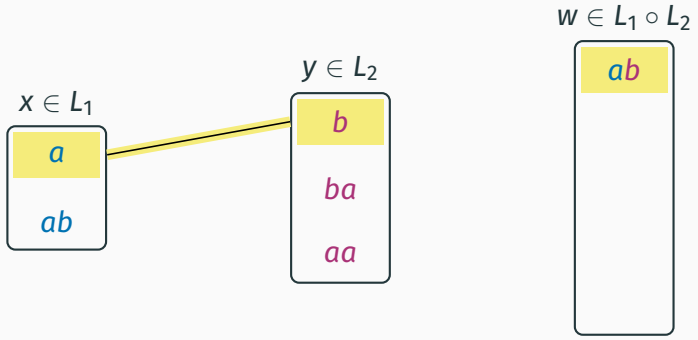
Concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.



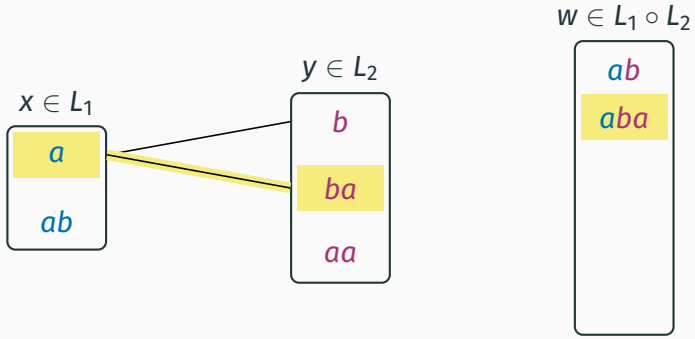
Concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.



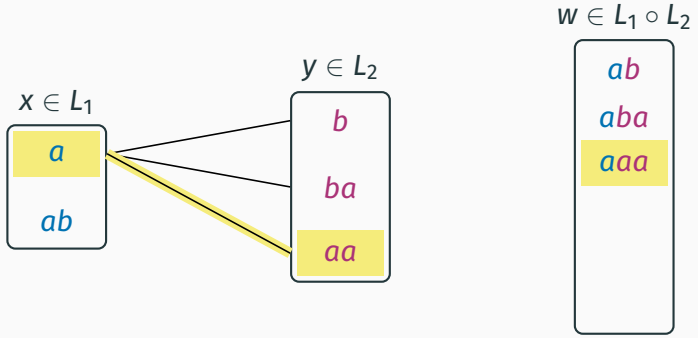
Concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.



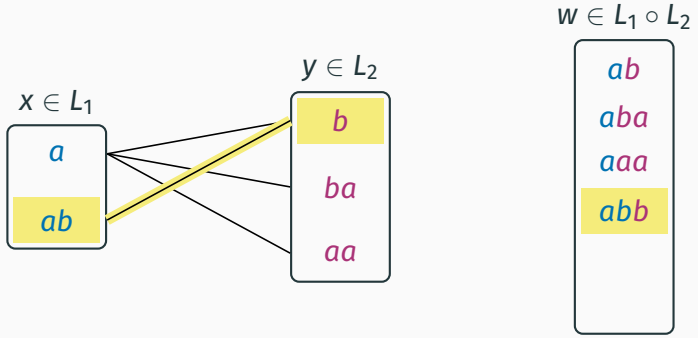
Concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.



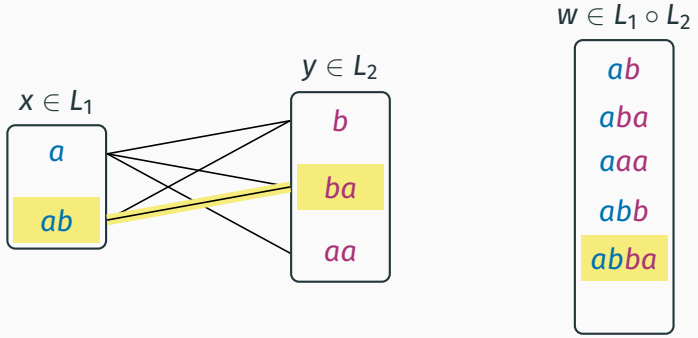
Concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.



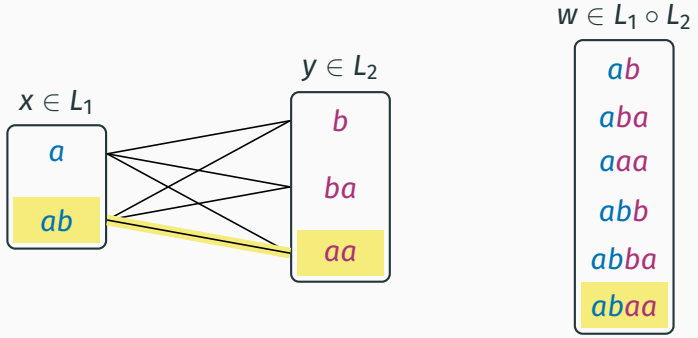
Concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.



Concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.



Decidable languages: concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.

Decidable languages: concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.

Theorem

If L_1 and L_2 are decidable, then $L_1 \circ L_2$ is decidable.

Decidable languages: concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.

Theorem

If L_1 and L_2 are decidable, then $L_1 \circ L_2$ is decidable.

$w =$ 

Brute force: try all splits $w = xy$.

- Accept if M_1 accepts x and M_2 accepts y.
- If no split works, reject.

Decidable languages: concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.

Theorem

If L_1 and L_2 are decidable, then $L_1 \circ L_2$ is decidable.

$w =$ | a | b | a | b |

$x = \varepsilon$ M_1 ✗

$y = abab$ M_2 ✓

Brute force: try all splits $w = xy$.

- Accept if M_1 accepts x and M_2 accepts y.
- If no split works, reject.

Decidable languages: concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.

Theorem

If L_1 and L_2 are decidable, then $L_1 \circ L_2$ is decidable.

$w =$ | a | b | a | b |

$x = a$ M_1 ✓

$y = bab$ M_2 ✗

Brute force: try all splits $w = xy$.

- Accept if M_1 accepts x and M_2 accepts y.
- If no split works, reject.

Decidable languages: concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.

Theorem

If L_1 and L_2 are decidable, then $L_1 \circ L_2$ is decidable.

$w =$ | a | b | a | b |

$x = ab$ M_1 ✓

$y = ab$ M_2 ✓

$\Rightarrow$ **accept**

Brute force: try all splits $w = xy$.

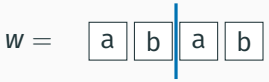
- Accept if M_1 accepts x and M_2 accepts y.
- If no split works, reject.

Decidable languages: concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.

Theorem

If L_1 and L_2 are decidable, then $L_1 \circ L_2$ is decidable.



$x =$ *ab* M_1 ✓

$y =$ *ab* M_2 ✓

$\Rightarrow$ **accept**

Brute force: try all splits $w = xy$.

- Accept if M_1 accepts x and M_2 accepts y.
- If no split works, reject.

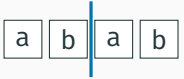
Question: how many splits in total?

Decidable languages: concatenation

For languages L_1 and L_2, their *concatenation* $L_1 \circ L_2$ is the set of all strings $w = xy$ with $x \in L_1$ and $y \in L_2$.

Theorem

If L_1 and L_2 are decidable, then $L_1 \circ L_2$ is decidable.

$w =$ 

$x = ab$ M_1 ✓

$y = ab$ M_2 ✓

$\Rightarrow$ **accept**

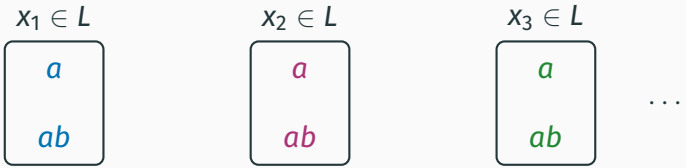
Brute force: try all splits $w = xy$.

- Accept if M_1 accepts x and M_2 accepts y.
- If no split works, reject.

Question: how many splits in total? $|w| + 1$

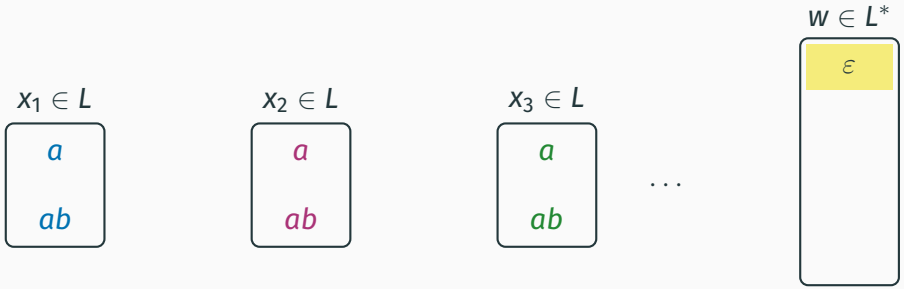
Kleene star

For a language L, its *Kleene star* L^* is the set of all strings $w = x_1 x_2 \cdots x_k$ with $k \geq 0$ and each $x_i \in L$.



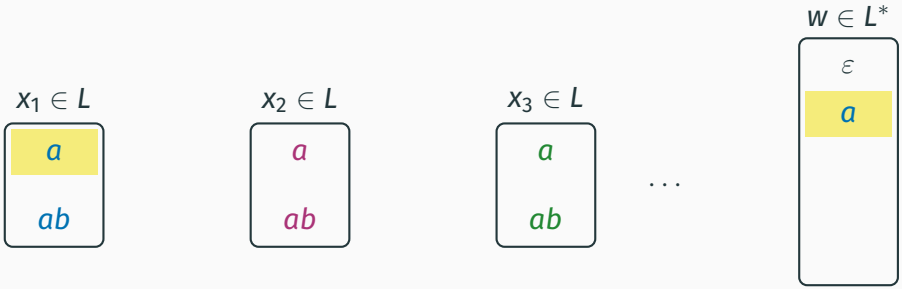
Kleene star

For a language L, its *Kleene star* L^* is the set of all strings $w = x_1 x_2 \cdots x_k$ with $k \geq 0$ and each $x_i \in L$.



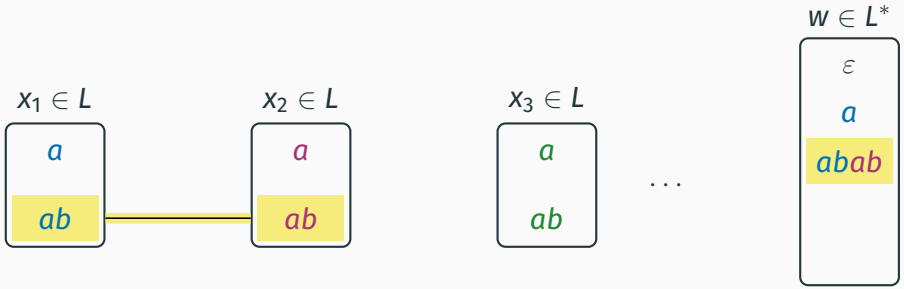
Kleene star

For a language L, its *Kleene star* L^* is the set of all strings $w = x_1 x_2 \cdots x_k$ with $k \geq 0$ and each $x_i \in L$.



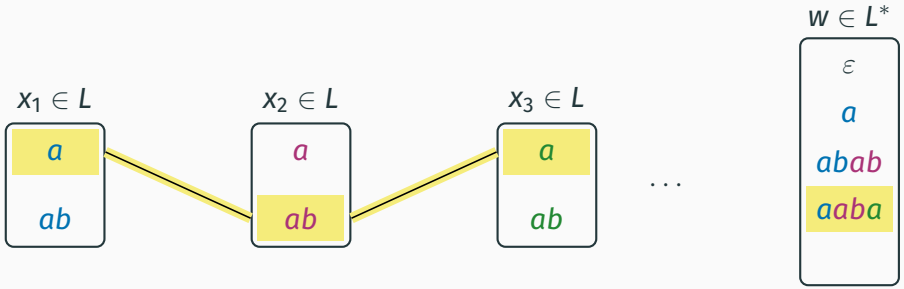
Kleene star

For a language L, its *Kleene star* L^* is the set of all strings $w = x_1 x_2 \cdots x_k$ with $k \geq 0$ and each $x_i \in L$.



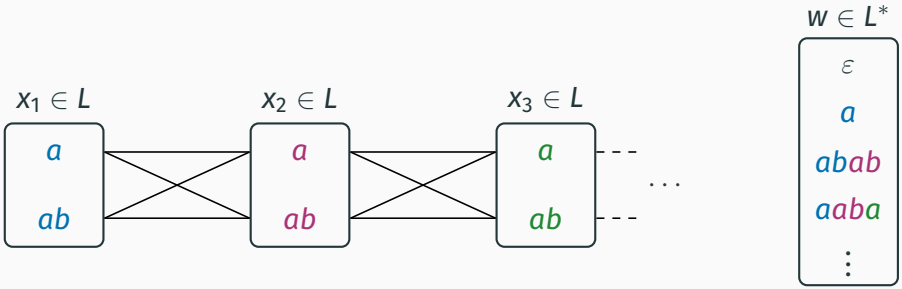
Kleene star

For a language L, its *Kleene star* L^* is the set of all strings $w = x_1 x_2 \cdots x_k$ with $k \geq 0$ and each $x_i \in L$.



Kleene star

For a language L, its *Kleene star* L^* is the set of all strings $w = x_1 x_2 \cdots x_k$ with $k \geq 0$ and each $x_i \in L$.



Kleene star: an example

Let $L = \{a,\ ab\}$. Recall $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

k	strings

Kleene star: an example

Let $L = \{a,\ ab\}$. Recall $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

k	strings
0	ε

Kleene star: an example

Let $L = \{a,\ ab\}$. Recall $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

k		strings
0		ε
1	x_1	$a,\ ab$

Kleene star: an example

Let $L = \{a,\ ab\}$. Recall $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

k		strings
0		ε
1	x_1	$a,\ ab$
2	$x_1 x_2$	$aa,\ aab,\ aba,\ abab$

Kleene star: an example

Let $L = \{a,\ ab\}$. Recall $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

k		strings
0		ε
1	x_1	$a,\ ab$
2	$x_1 x_2$	$aa,\ aab,\ aba,\ abab$
3	$x_1 x_2 x_3$	$aaa,\ aaab,\ aaba,\ aabab,$ $abaa,\ abaab,\ ababa,\ ababab$

Kleene star: an example

Let $L = \{a,\ ab\}$. Recall $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

k		strings
0		ε
1	x_1	$a,\ ab$
2	$x_1 x_2$	$aa,\ aab,\ aba,\ abab$
3	$x_1 x_2 x_3$	$aaa,\ aaab,\ aaba,\ aabab,$ $abaa,\ abaab,\ ababa,\ ababab$
$\vdots$		$\vdots$

$L^* = \{\varepsilon,\ a,\ aa,\ ab,\ aaa,\ aab,\ aba,\ \ldots\}$

Decidable languages: Kleene star

Reminder: $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

Decidable languages: Kleene star

Reminder: $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

Theorem

If L is decidable, then L^* is decidable.

Decidable languages: Kleene star

Reminder: $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

Theorem

If L is decidable, then L^* is decidable.

Your turn: given a decider M for L, design a decider for L^*.

Hint: brute force is allowed.

Decidable languages: Kleene star

Reminder: $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

Theorem

If L is decidable, then L^* is decidable.

$w =$ | a | b | a | b |

Brute force: if $w = \varepsilon$, accept. Otherwise, for each $k = 1, \ldots, |w|$, try each split $w = x_1 x_2 \cdots x_k$ into k pieces.

- Accept if M accepts every x_i.
- If no split works, reject.

Decidable languages: Kleene star

Reminder: $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

Theorem

If L is decidable, then L^* is decidable.

$w =$ | a | b | a | b |

split: $abab$ $(k = 1)$

M: ✗

Brute force: if $w = \varepsilon$, accept. Otherwise, for each $k = 1, \ldots, |w|$, try each split $w = x_1 x_2 \cdots x_k$ into k pieces.

- Accept if M accepts every x_i.
- If no split works, reject.

Decidable languages: Kleene star

Reminder: $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

Theorem

If L is decidable, then L^* is decidable.

$w =$ | a | b | a | b |

split: $a \mid bab$ $(k = 2)$

M: ✓ ✗

Brute force: if $w = \varepsilon$, accept. Otherwise, for each $k = 1, \ldots, |w|$, try each split $w = x_1 x_2 \cdots x_k$ into k pieces.

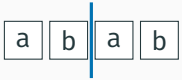
- Accept if M accepts every x_i.
- If no split works, reject.

Decidable languages: Kleene star

Reminder: $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

Theorem

If L is decidable, then L^* is decidable.

$w =$ 

split: $ab \mid ab$ $(k = 2)$

M: ✓ ✓

$\Rightarrow$ **accept**

Brute force: if $w = \varepsilon$, accept. Otherwise, for each $k = 1, \ldots, |w|$, try each split $w = x_1 x_2 \cdots x_k$ into k pieces.

- Accept if M accepts every x_i.
- If no split works, reject.

Decidable languages: Kleene star

Reminder: $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

Theorem

If L is decidable, then L^* is decidable.

$w =$ | a | b | a | b |

split: $ab \mid ab$ $(k = 2)$

M: ✓ ✓

$\Rightarrow$ **accept**

Brute force: if $w = \varepsilon$, accept. Otherwise, for each $k = 1, \ldots, |w|$, try each split $w = x_1 x_2 \cdots x_k$ into k pieces.

- Accept if M accepts every x_i.
- If no split works, reject.

Question: how many splits in total?

Decidable languages: Kleene star

Reminder: $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in L\}$.

Theorem

If L is decidable, then L^* is decidable.



Brute force: if $w = \varepsilon$, accept. Otherwise, for each $k = 1, \ldots, |w|$, try each split $w = x_1 x_2 \cdots x_k$ into k pieces.

- Accept if M accepts every x_i.
- If no split works, reject.

Question: how many splits in total? $2^{|w|-1}$

Decidable languages: complement

For a language L, its *complement* L^c is the set of all strings that are *not* in L.

Decidable languages: complement

For a language L, its *complement* L^c is the set of all strings that are *not* in L.

Theorem

If L is decidable, then L^c is decidable.

Decidable languages: complement

For a language L, its *complement* L^c is the set of all strings that are *not* in L.

Theorem

If L is decidable, then L^c is decidable.

Let M be a decider for L. We construct a decider M' for L^c. On input w:

1. Run M on w.

Decidable languages: complement

For a language L, its *complement* L^c is the set of all strings that are *not* in L.

Theorem

If L is decidable, then L^c is decidable.

Let M be a decider for L. We construct a decider M' for L^c. On input w:

1. Run M on w.
2. If M accepts, M' rejects;
   if M rejects, M' accepts.

Closure properties so far

	intersection	union	concatenation	Kleene star	complement
regular	✓	✓	✓	✓	✓
decidable	✓	✓	✓	✓	✓
recognizable	?	?	?	?	?

Recognizable languages

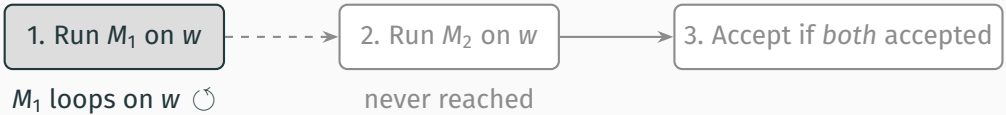
Recognizable languages: intersection

Now M_1 and M_2 are only *recognizers*. Run them one after the other:



Recognizable languages: intersection

Now M_1 and M_2 are only *recognizers*. Run them one after the other:



Recognizable languages: intersection

Now M_1 and M_2 are only *recognizers*. Run them one after the other:



Then $w \notin L_1$, so $w \notin L_1 \cap L_2$, and a recognizer may loop on non-members.

Recognizable languages: union?

Now M_1 and M_2 are only *recognizers*. Run them one after the other:



Recognizable languages: union?

Now M_1 and M_2 are only *recognizers*. Run them one after the other:



Recognizable languages: union?

Now M_1 and M_2 are only *recognizers*. Run them one after the other:



Then M_2 never runs, so w is never accepted, even if $w \in L_2$.

Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup$ = "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

Correctness: $M_\cup$ accepts w if and only if $w \in L_1 \cup L_2$

Case 1 (M_1 or M_2 accepts w): $M_\cup$ accepts

Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup$ = "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

Correctness: $M_\cup$ accepts w if and only if $w \in L_1 \cup L_2$

Case 1 (M_1 or M_2 accepts w): $M_\cup$ accepts

Case 2 (both reject w): $M_\cup$ rejects

Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup$ = "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

Correctness: $M_\cup$ accepts w if and only if $w \in L_1 \cup L_2$

Case 1 (M_1 or M_2 accepts w): $M_\cup$ accepts

Case 2 (both reject w): $M_\cup$ rejects

Case 3 (neither accepts w, and at least one loops): $M_\cup$ loops

QED

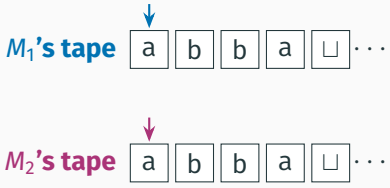
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup$ = "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

start: w on both tapes



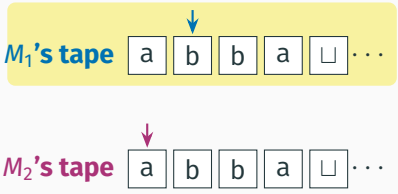
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup$ = "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

turn 1: one step of M_1



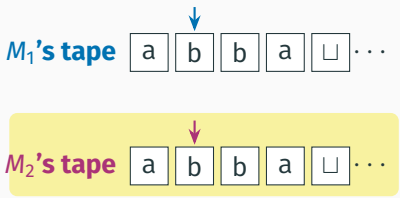
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

turn 2: one step of M_2



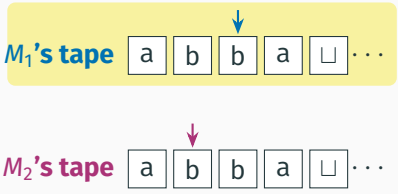
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup$ = "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

turn 3: one step of M_1



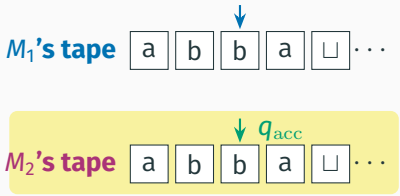
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup$ = "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

turn 4: one step of M_2



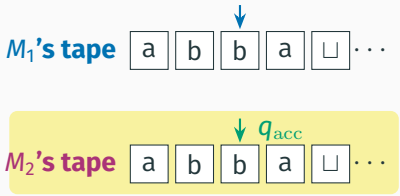
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

turn 4: one step of M_2



M_2 has accepted $\Rightarrow$ accept w

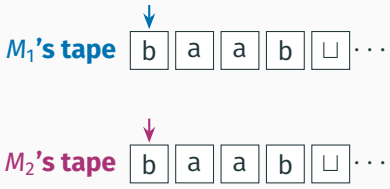
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup$ = "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

another input w'



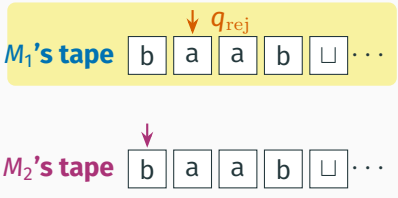
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

turn 1: one step of M_1



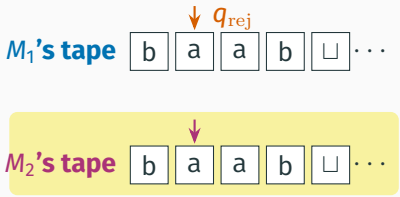
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

turn 2: one step of M_2



M_1 **has rejected, so** $w' \notin L_1$

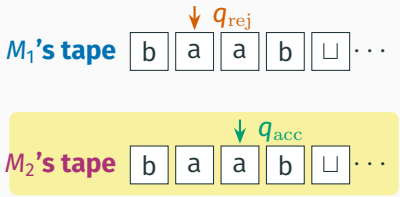
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup$ = "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

turn 3: M_1 has halted, so M_2 again



M_1 has rejected, so $w' \notin L_1$

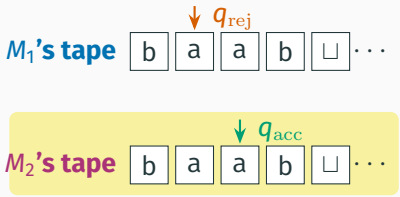
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup$ = "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

turn 3: M_1 has halted, so M_2 again



M_2 has accepted $\Rightarrow$ accept w

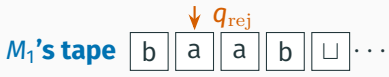
Recognizable languages: union

Run M_1 and M_2 *in parallel*: take turns until one accepts or both reject.

$M_\cup$ = "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, accept.
7. If both have rejected, reject."

turn 3: M_1 has halted, so M_2 again



M_2 has accepted $\Rightarrow$ accept w

Reminiscent of *round-robin* scheduling?

Recognizable languages: concatenation?

Now M_1 and M_2 are only *recognizers*. Try the splits of $w = abab$ one at a time:



Recognizable languages: concatenation?

Now M_1 and M_2 are only *recognizers*. Try the splits of $w = abab$ one at a time:

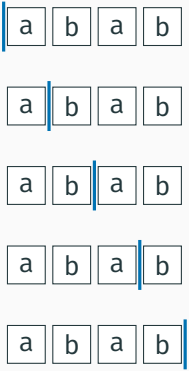


Then $ab \mid ab$ is never tried, even if $ab \in L_1$ and $ab \in L_2$.

Recognizable languages: concatenation

Reminder: $L_1 \circ L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

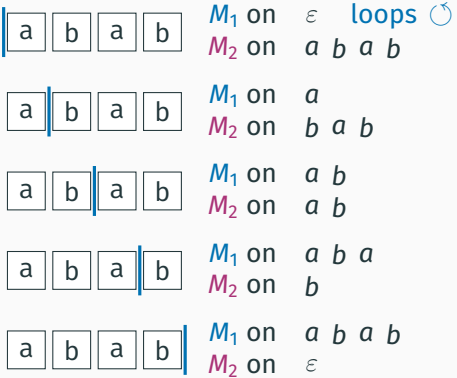
All possible ways to split $w = abab$ into $x \mid y$:



Recognizable languages: concatenation

Reminder: $L_1 \circ L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

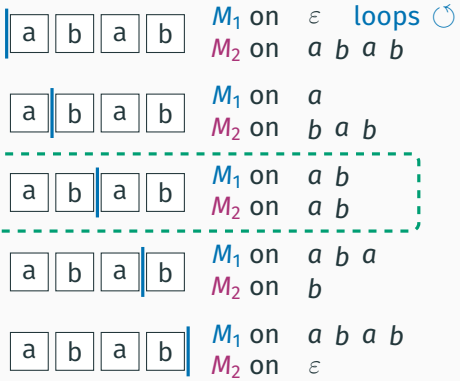
Each split gives two *runs*: M_1 on x, and M_2 on y.



Recognizable languages: concatenation

Reminder: $L_1 \circ L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

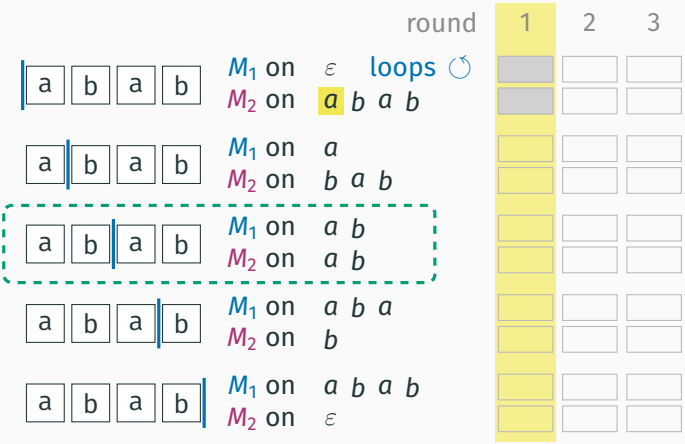
Goal: find a split where *both* runs accept, here $ab \mid ab$.



Recognizable languages: concatenation

Reminder: $L_1 \circ L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

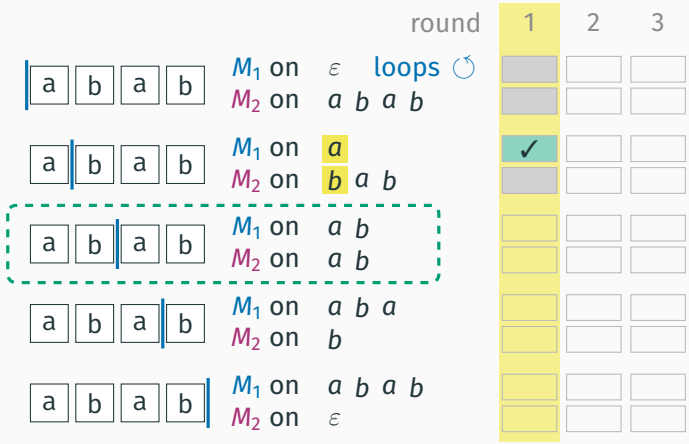
Run them *in parallel*: in each round, every *unfinished* run takes one step.



Recognizable languages: concatenation

Reminder: $L_1 \circ L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

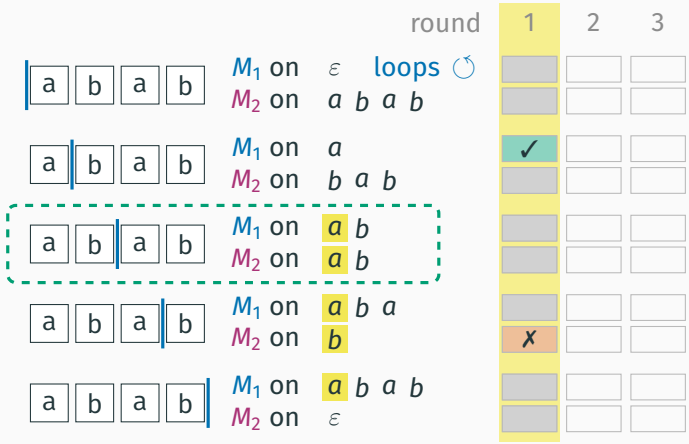
Run them *in parallel*: in each round, every *unfinished* run takes one step.



Recognizable languages: concatenation

Reminder: $L_1 \circ L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

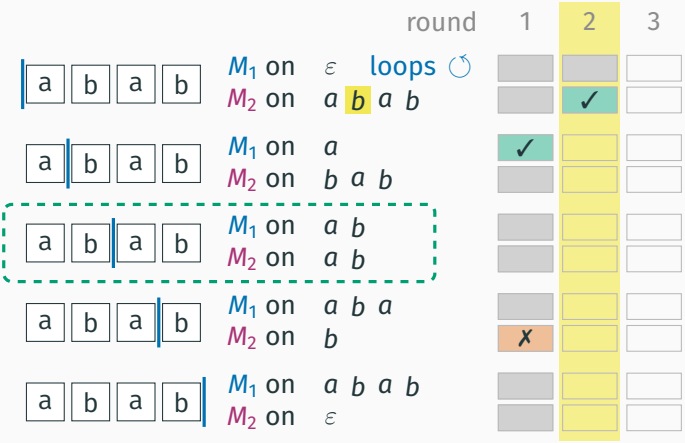
Run them *in parallel*: in each round, every *unfinished* run takes one step.



Recognizable languages: concatenation

Reminder: $L_1 \circ L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

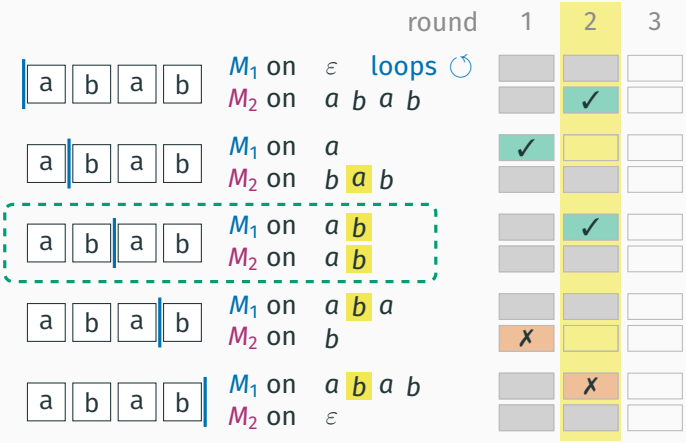
Run them *in parallel*: in each round, every *unfinished* run takes one step.



Recognizable languages: concatenation

Reminder: $L_1 \circ L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

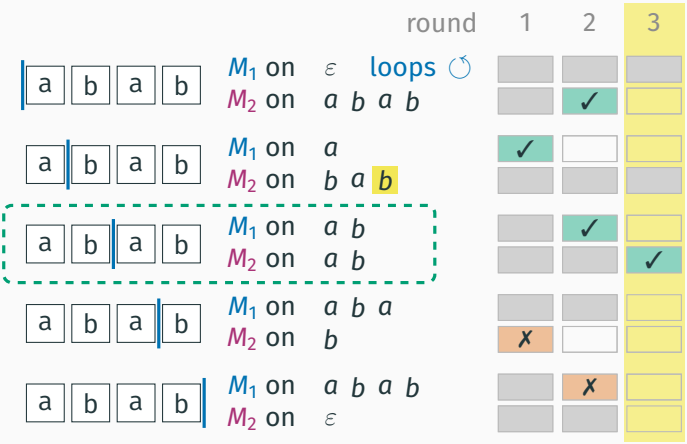
Run them *in parallel*: in each round, every *unfinished* run takes one step.



Recognizable languages: concatenation

Reminder: $L_1 \circ L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

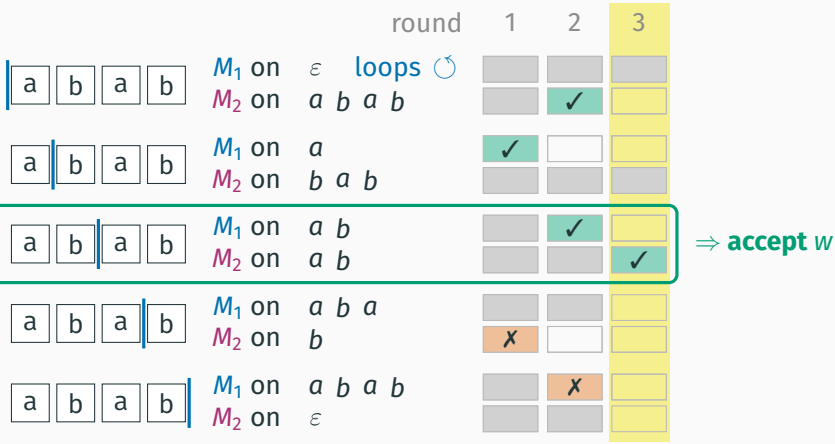
Run them *in parallel*: in each round, every *unfinished* run takes one step.



Recognizable languages: concatenation

Reminder: $L_1 \circ L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

Run them *in parallel*: in each round, every *unfinished* run takes one step.



Closure properties so far

	intersection	union	concatenation	Kleene star	complement
regular	✓	✓	✓	✓	✓
decidable	✓	✓	✓	✓	✓
recognizable	✓	✓	✓	✓	**?**

Closure properties so far

	intersection	union	concatenation	Kleene star	complement
regular	✓	✓	✓	✓	✓
decidable	✓	✓	✓	✓	✓
recognizable	✓	✓	✓	✓	?

key idea: run the machines *in parallel* (take turns)

Recognizable languages: complement?

Try the construction for deciders on a *recognizer*.

Let M be a recognizer for L. We construct M' as before. On input w:

1. Run M on w.
2. If M accepts, M' rejects;
   if M rejects, M' accepts.

Recognizable languages: complement?

Try the construction for deciders on a *recognizer*.

Let M be a recognizer for L. We construct M' as before. On input w:

1. Run M on w. $\leftarrow$ may loop forever if $w \notin L$ $\circlearrowleft$
2. If M accepts, M' rejects;
   if M rejects, M' accepts.

Recognizable languages: complement?

Try the construction for deciders on a *recognizer*.

Let M be a recognizer for L. We construct M' as before. On input w:

1. Run M on w. $\leftarrow$ may loop forever if $w \notin L$ $\circlearrowleft$
2. If M accepts, M' rejects;
   if M rejects, M' accepts.

If M loops on w, then so does M': it never accepts w, even though $w \in L^c$.

Recognizable languages: complement ✗

Recall: a class of languages is

- *closed* under complement if for *every* L in the class, L^c is also in the class;
- *not closed* under complement if for *some* L in the class, L^c is not in the class.

Recognizable languages: complement ✗

Recall: a class of languages is

- *closed* under complement if for *every* L in the class, L^c is also in the class;
- *not closed* under complement if for *some* L in the class, L^c is not in the class.

Theorem

Recognizable languages are not closed under complement.

Recognizable languages: complement ✗

Recall: a class of languages is

- *closed* under complement if for *every* L in the class, L^c is also in the class;
- *not closed* under complement if for *some* L in the class, L^c is not in the class.

Theorem

Recognizable languages are not closed under complement.

We only need to find a recognizable L whose complement L^c is *not* recognizable.

Recognizable languages: complement ✗

Recall: a class of languages is

- *closed* under complement if for *every* L in the class, L^c is also in the class;
- *not closed* under complement if for *some* L in the class, L^c is not in the class.

Theorem

Recognizable languages are not closed under complement.

We only need to find a recognizable L whose complement L^c is *not* recognizable.
Candidate: A_{TM}, which is recognizable but undecidable (Lecture 20).

Recognizable languages: complement ✗

Recall: a class of languages is

- *closed* under complement if for *every* L in the class, L^c is also in the class;
- *not closed* under complement if for *some* L in the class, L^c is not in the class.

Theorem

Recognizable languages are not closed under complement.

We only need to find a recognizable L whose complement L^c is *not* recognizable.
Candidate: A_{TM}, which is recognizable but undecidable (Lecture 20).

Claim

$(A_{\mathrm{TM}})^c$ is not recognizable.

Recognizable languages: complement ✗

Recall: a class of languages is

- *closed* under complement if for *every* L in the class, L^c is also in the class;
- *not closed* under complement if for *some* L in the class, L^c is not in the class.

Theorem

Recognizable languages are not closed under complement.

We only need to find a recognizable L whose complement L^c is *not* recognizable.
Candidate: A_{TM}, which is recognizable but undecidable (Lecture 20).

Claim

$(A_{\mathrm{TM}})^c$ is not recognizable.

To prove it, we prove something more general.

The main theorem

Theorem

L is decidable $\iff$ both L and L^c are recognizable.

The main theorem

Theorem

L is decidable $\iff$ both L and L^c are recognizable.

Claim

$(A_{\mathrm{TM}})^c$ is not recognizable.

Proof of the claim

Recall: “$P \Rightarrow Q$” is equivalent to its *contrapositive* “not $Q \Rightarrow$ not P”.

Proof of the claim

Recall: "$P \Rightarrow Q$" is equivalent to its *contrapositive* "not $Q \Rightarrow$ not P".

Theorem ($\Leftarrow$)

If L and L^c are both recognizable, then L is decidable.

Proof of the claim

Recall: "$P \Rightarrow Q$" is equivalent to its *contrapositive* "not $Q \Rightarrow$ not P".

Theorem ($\Leftarrow$)

If L and L^c are both recognizable, then L is decidable.

Contrapositive

If L is not decidable, then L or L^c is not recognizable.

Proof of the claim

Recall: "$P \Rightarrow Q$" is equivalent to its *contrapositive* "not $Q \Rightarrow$ not P".

Theorem ($\Leftarrow$)

If L and L^c are both recognizable, then L is decidable.

Contrapositive

If L is not decidable, then L or L^c is not recognizable.

- A_{TM} is not decidable, so A_{TM} or $(A_{\mathrm{TM}})^c$ is not recognizable.

Proof of the claim

Recall: "$P \Rightarrow Q$" is equivalent to its *contrapositive* "not $Q \Rightarrow$ not P".

Theorem ($\Leftarrow$)

If L and L^c are both recognizable, then L is decidable.

Contrapositive

If L is not decidable, then L or L^c is not recognizable.

- A_{TM} is not decidable, so A_{TM} or $(A_{\mathrm{TM}})^c$ is not recognizable.
- A_{TM} is recognizable, so $(A_{\mathrm{TM}})^c$ is not recognizable.

Proof of the main theorem

($\Rightarrow$) If L is decidable, then L and L^c are both recognizable.

Proof of the main theorem

($\Rightarrow$) If L is decidable, then L and L^c are both recognizable.

Proof: L^c is decidable too, and every decider is a recognizer.

Proof of the main theorem

($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof of the main theorem

($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

Proof of the main theorem

($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

Correctness: M halts on every w and accepts exactly the strings of L.

Case 1 ($w \in L$): M_1 accepts w and M_2 does not, so M accepts

Proof of the main theorem

($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

Correctness: M halts on every w and accepts exactly the strings of L.

Case 1 ($w \in L$): M_1 accepts w and M_2 does not, so M accepts

Case 2 ($w \in L^c$): M_2 accepts w and M_1 does not, so M rejects

QED

Proof of the main theorem

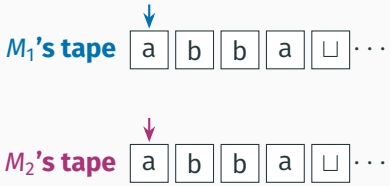
($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

start: w on both tapes



Proof of the main theorem

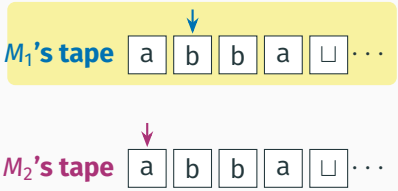
($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

turn 1: one step of M_1



Proof of the main theorem

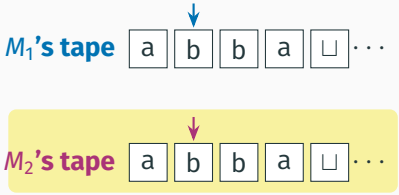
($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

turn 2: one step of M_2



Proof of the main theorem

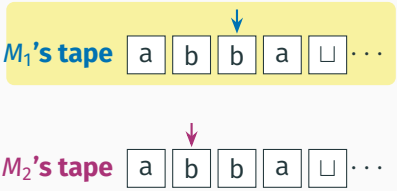
($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

turn 3: one step of M_1



Proof of the main theorem

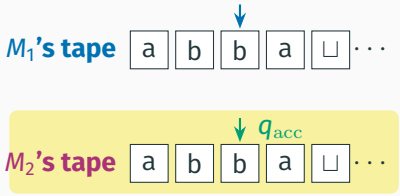
($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

turn 4: one step of M_2



Proof of the main theorem

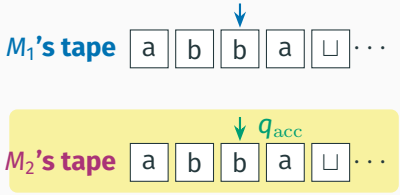
($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

turn 4: one step of M_2



M_2 **has accepted** $\Rightarrow$ **reject** w

Proof of the main theorem

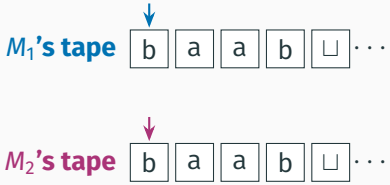
($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

another input w'



Proof of the main theorem

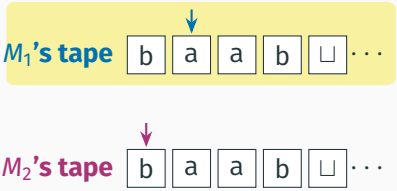
($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

turn 1: one step of M_1



Proof of the main theorem

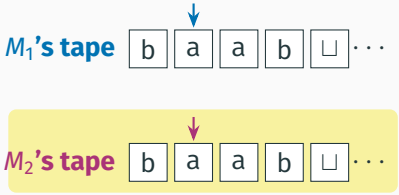
($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

turn 2: one step of M_2



Proof of the main theorem

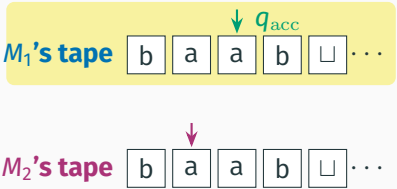
($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

turn 3: one step of M_1



Proof of the main theorem

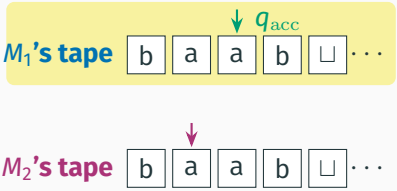
($\Leftarrow$) If L and L^c are both recognizable, then L is decidable.

Proof: let M_1 recognize L and M_2 recognize L^c. Run them *in parallel*:

$M =$ "On input w:

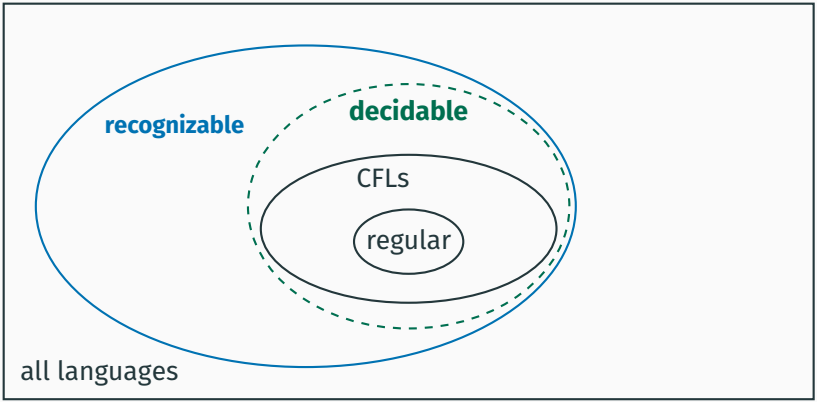
1. Copy w onto both tapes.
2. Repeat:
3. If M_1 has not halted, run it for one step.
4. If M_1 has accepted, accept.
5. If M_2 has not halted, run it for one step.
6. If M_2 has accepted, reject."

turn 3: one step of M_1



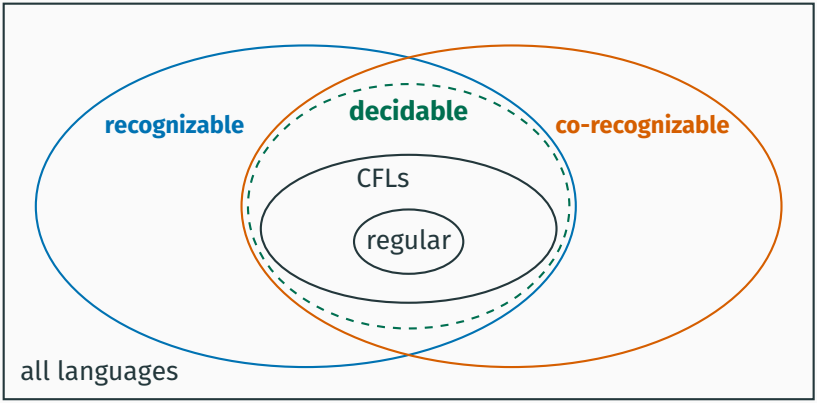
M_1 **has accepted** $\Rightarrow$ **accept** w

Classes of languages



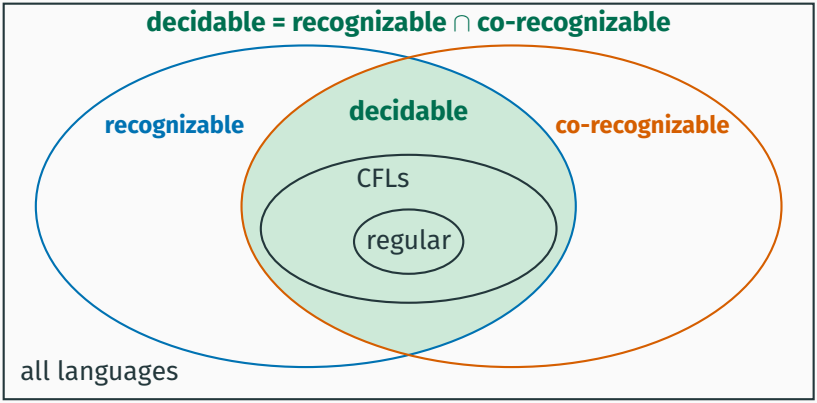
Classes of languages

L is *co-recognizable* if its complement L^c is recognizable.



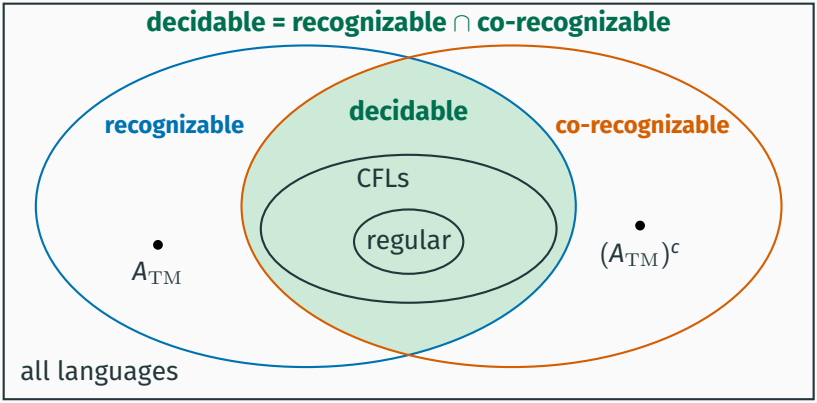
Classes of languages

L is *co-recognizable* if its complement L^c is recognizable.



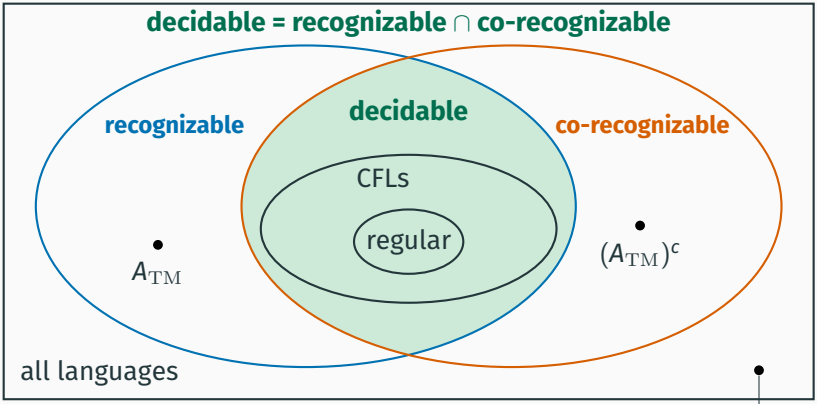
Classes of languages

L is *co-recognizable* if its complement L^c is recognizable.



Classes of languages

L is *co-recognizable* if its complement L^c is recognizable.



neither recognizable nor co-recognizable, e.g. *TM equivalence*: given two TMs, do they accept the same language? (Sipser Thm 5.30)

Summary

	intersection	union	concatenation	Kleene star	complement
decidable	✓	✓	✓	✓	✓
recognizable	✓	✓	✓	✓	✗

- **Recurring key idea:** recognizers may loop, so run them *in parallel*.
- **Theorem:** L is decidable $\iff$ both L and L^c are recognizable.